

AutoResearch 专家线下标注教程 | PCA 双镜像对齐版

参考格式：（starter / tests / reference/ public_assets格式不固定 根据优化面需要设置）



population-genetics-pca_teaching-...
299.32KB



2026-09-29 修订：PCA 双镜像对齐版。保留四个阶段、研究质量门槛、模型组合、时长及证据模板；本次调整双镜像封装、可见性与最终评分链路。

每轮：Agent 修改方法 → 在 Agent 环境运行 Public/Dev 评分 → 根据反馈继续迭代。每次 Agent 运行结束后：Harbor 移交最终候选 → 独立 Verifier 执行 Hidden 评分。Verifier 运行评分与约束检查，并按协议执行候选代码或加载模型，不启动第二个 Coding Agent；本版不要求每轮启动独立评分容器或新增评分服务。

原始教程已有 Hidden 隔离；此次主要补齐第二套 Dockerfile、独立构建上下文、提交物传递与实际评分。Hidden 不是单靠新增目录实现隔离。[原始教程](#)



注意⚠：由于 **PCA 双镜像对齐版**要求 Verifier 与 Agent 在迭代过程中独立构建 container，AutoDL 等基于 Docker 的容器云服务器**无法启动 Harbor trials 进行验证**。因此，请慎用 AutoDL 等容器云；可租用阿里云等服务器，或先构建合格的 Harbor 任务包并在非容器云环境中测试，之后再在容器云服务器上进行轨迹迭代。此处不限制使用的具体平台，如下为之前的出题专家推荐性价比较高的平台：<https://www.compshare.cn/>
<https://ai.blsc.cn/>

内容门槛	通过依据	退回与补充证据
G01 方法优化面	题面与实际接口允许提出并实现方法更新，例如目标形式、采样/更新规则、模型模块或推理策略。固定任务只搜索学习率、训练轮数、loss/ridge 权重的一组数字不属于本批次接收题型。	同时检查 instruction、候选解析/guard 与代码调用链；不能只改措辞。应开放真实方法接口并保留评分约束。方法接口开放但 Reference 只改参数时，补充非纯调参方向及空间证据，不仅凭 diff 判整题失败。

G02 合理 Baseline	可正常运行、来源可解释，数据、评测、seed 和预算可比。默认取未修改 Starter；若 Starter 仅为 Scaffold，可不给 method，但需提交专家侧可执行朴素 Baseline 及评分锚点映射。	核对实际 steps/tokens/样本量、训练曲线、早停/失败原因、checkpoint、调参机会和完整 seed。只有直接证明不公平削弱或无代表性的差对照才不通过；简单、经典或年代较旧不能单独判退。
G03 提升充分性	同协议全部正式成对运行有效、真实正向改善，Reference 归一化在 [0.15,0.8]。随机评估采用 Baseline 样本标准差，改善至少 $3\sigma_B$ ； $3-5\sigma_B$ 可接受并建议复核， $\geq 5\sigma_B$ 为强证据。	复算逐 seed 值、均值、 $n-1$ 样本标准差、质量门和归一化。确定性评估不设通用 5% 门槛；题目预先声明其他容差时同时核验。不能通过修改锚点 U 制造达标。

Baseline 训练充分性与方法代表性

- 训练不足看实际训练量和停止原因，不只看配置里的 epochs。仅给 Baseline 减训练、减数据或禁用关键输入，Reference 使用完整预算且不是允许研究变量时，不通过。有限预算下双方按同一规则停止，不因未完全收敛自动判退。
- 若研究训练效率，按同计算、时间或 token 预算比较，不能机械强求同 epochs；架构或模型容量可改时，同容量也不是一律要求。声明研究变量并提供预算匹配证据。
- “旧方法不合理”须有任务适配、资源可行、同协议可比的常用更强锚点与可核验来源。没有可比证据时写待补证，不能凭年代或对 SOTA 的印象判退。
- 已批准的算法 debug 题须明确故障修复目标并提供健康对照，不能把已知故障制造的分差当作一般方法创新收益。
- 区分独立训练 seed 与同一模型随机评估的 replicate。合法共享预训练起点可以声明后使用；复制已完成 checkpoint 不能冒充多次独立训练。

Docker 路径契约：对齐 PCA 内层双镜像任务

以例题内层的 task.toml、environment/Dockerfile、tests/Dockerfile 和 tests/test.sh 为封装依据；不把外层包装目录当作原生双镜像任务。例题提交路径是 /app/submission；本教程保留原 /workspace/solution，所有配置、入口和复制路径统一使用后者。

对象	严格约定
提交包任务根	workspace/harbor_task/，是平台完整任务包，不整体挂载或复制给 Agent。
Agent 镜像	environment/Dockerfile；build context 为 environment/。只包含公开依赖、Starter、Public/Dev 数据和评分入口。

Verifier 镜像	tests/Dockerfile；build context 为 tests/。自带 /tests/test.sh、可信 grader、评分依赖与受保护评测资产。
候选路径	Agent 中 /workspace/solution；结束后通过 artifacts 移交到 Verifier 的同一路径。需要的模型和配置也必须包含在提交契约内。
COPY/ADD	Agent 的源路径相对 environment/，例如 starter/、public_eval/；Verifier 的源路径相对 tests/。禁止 COPY ../，不再混用相对任务根的旧规则。
正式评分入口	Verifier 内 bash /tests/test.sh；读取 /workspace/solution，实际执行评分并写 /logs/verifier/reward.txt 或数值对象 reward.json。不假设启动 cwd。

代码块

```
1  # Agent 镜像: context 是 environment/
2  docker build -t task-agent:local \
3    -f workspace/harbor_task/environment/Dockerfile \
4    workspace/harbor_task/environment
5
6  # Verifier 镜像: context 是 tests/
7  docker build -t task-verifier:local \
8    -f workspace/harbor_task/tests/Dockerfile \
9    workspace/harbor_task/tests
10
```

两套镜像在运行前构建。Agent 的每轮 Public/Dev 评分不重复构建镜像。原 example.zip 附件保留作旧布局参考，其中单镜像 COPY 规则、占位配置及仅运行 pytest 的 test.sh 不作为本版实现依据；本版需要两套环境实际构建、正式入口实际评分。静态检查或文档示例不能替代目标 Harbor 版本的实跑记录。

统一时长、报告与专家退回说明

- 两种模型组合分别有效执行至少 10h；各至少 7h 的例外仅适用于不涉及训练且单轮迭代很短的任务，并须满足本教程全部例外证据条件。两条不相加，排队、安装、构建故障与长时间阻塞不计。启动词中的旧“12h且大于15轮”不再作为统一门槛。

- 容器应能稳定存活至少 12h，是环境稳定性目标，独立于 Agent 的有效迭代时长；只读质检不能声称已完成此压力测试。
- 报告开头依次写优化面、Baseline 方法、Reference 方法与全部正式 seed 成对跑分、G01/G02/G03 判定，随后写两条轨迹、格式和既有21项检查。
- 每个失败或待补证项写“问题事实及文件位置 → 不满足原因 → 具体修改动作 → 重交验收材料”。例如：只能提交权重字典且禁止函数，无法实现方法更新；请开放方法接口并同步 guard，重交题面、方法差异与同 seed 对照证据。
- 自动静态质检只覆盖已声明的检查范围。QA07/08仅题面泄露、QA15暂不检查平台资源上限；完整物理隔离、正式 Hidden 运行及资源/稳定性仍由平台完成，不因本地静态通过而豁免。

1. 登录并领取本地出题任务

本阶段的目标是确认任务输入、出题模式和资源边界，再开始实现。领取后不要直接改代码，先核对任务卡中的要求是否完整、材料是否可访问。

出题模式与输入

模式	专家收到的输入	需要完成的工作
S1 强约束	候选论文和预置代码库	优先选择有公开源码、相对 Baseline 有明确可测提升且资源可压缩的方向；抽取可验证的优化面并整理为问题定义、环境与评分校验。无需复现整篇论文，但必须提供一个合法、可复现的 Reference Solution。
S2 半开放	研究方向或题面种子，以及参考材料、模型和数据白名单	自行确定搜索空间、Baseline、难度、评分协议和完整题目。
S3 全开放	已通过平台确认的 Proposal	按确认后的任务说明实现；进入实现阶段后，交付格式与 S2 相同。



平台给出的优化面只用于参考。专家应根据论文和代码重新判断：题目是否能程序化评分、一次迭代能否压缩到可接受范围、Reference Solution 是否未饱和，以及是否仍有题面未直接提示的合法改进空间。

领取后检查

- ☐ 任务输入可以正常下载，代码和数据的使用范围明确
- ☐ 已确认主指标名称及 maximize 或 minimize 方向

- ☐ 已确认 GPU、CPU、内存、网络 and 单次迭代限制
- ☐ 已确认 Starter Code、Public/Dev、Hidden Test 和 Reference Solution 的可见性边界
- ☐ 已确认最终需要提交 workspace/、expert_evidence/ 与 optimization_evidence/，并明确三者的可见性边界

题目变体

如果任务要求制作变体，每个变体都按一道独立题目交付。变体应改变模型架构、优化目标、起始代码或限定搜索方向，并引出本质不同的 Reference Solution。每个变体都要独立运行 Baseline、Reference、评分校验、随机性和资源检查。只改标题、随机 seed 或无实质影响的参数不算有效变体。

2. 构建 Repo

最终 Repo 只保留任务运行所需的标准目录。先完成任务定义，再搭建环境和评测，最后加入可运行解法。路径、文件名和可见性边界必须保持一致。

Repo 结构

代码块

```
1  workspace/
2  |—— harbor_task/                # 平台完整任务包；不整体暴露给 Agent
3  |   |—— instruction.md          # Harness 提供给 Agent 的任务说明
4  |   |—— task.toml              # separate、artifacts、资源与超时
5  |   |—— environment/           # Agent 镜像构建上下文
6  |   |   |—— Dockerfile
7  |   |   |—— requirements.txt
8  |   |   |—— starter/           # 原起始实现，如 method.py、solve.sh
9  |   |   |—— public_eval/       # 从旧 tests 拆出的公开 Dev 评分代码
10 |   |   |   |—— grader.py      # 内部模块按题目需要设置
11 |   |   |   |—— public_assets/
12 |   |   |   |—— train/         # 可选训练数据
13 |   |   |   |—— dev/          # 公开迭代评测，不用于训练
14 |   |   |   |—— 其他公开离线资源
15 |   |—— tests/                 # 仅 Verifier 可见的构建上下文
16 |   |   |—— Dockerfile        # 新增：独立评分镜像
17 |   |   |—— requirements.txt  # 评分依赖，按需提供
18 |   |   |—— test.sh           # 最终 Hidden 评分入口
19 |   |   |—— grader.py         # 可信正式评分实现
20 |   |   |—— 其他评分与约束检查模块
21 |   |   |—— hidden_assets/    # 可选；仅私有侧预置或安全注入
22 |—— reference/                # 专家侧参考解，Agent 不可见
```

源码目录与运行目录的区别： environment/starter/ 同时用于初始化只读 /workspace/starter 和候选工作区 /workspace/solution。本文后续的 solution/ 均指容器内这一候选工作区；不再把仓库根 harbor_task/solution/ 当作 Agent 构建输入，也不与 Harbor 可选 Oracle 解法目录混用。 public_eval/ 为本教程命名，可按任务已有公开接口等价组织。

区域	Agent 权限	用途
harbor_task/ 完整包	不整体可见	平台分别构建两套环境；Harness 提供公开任务说明。
/workspace/solution/	按题面可写	唯一候选提交面；本示例允许修改 method.py，solve.sh 为冻结入口。
/workspace/starter、public_eval、public_assets	按题面只读	起始实现、Public/Dev 评分与公开数据；不含 Hidden 或参考实现。
tests/、/tests/（最终 Verifier 私有目录）	不可见	独立 Verifier 中的评分、约束检查及私有资产。
reference/	不可见	专家参考解与难度标定。
expert_evidence/、optimization_evidence/ 完整包	不可见	专家交付和成对证据，只交平台；不挂载给 Agent。

统一术语

术语	本教程中的唯一含义
Starter Code	environment/starter/ 中的只读起始代码。
Baseline	默认是未经修改、可运行 Starter 在声明正式协议下的结果 B。若 Starter 仅提供 Scaffold、不给 method，专家须另交可执行的合理朴素 Baseline，并说明源码、入口及与初始方案的映射。主评分零点 B 必须来自该正式 Baseline 的真实聚合结果，不能另设更弱锚点制造提升。
solution/	容器内 /workspace/solution，从 environment/starter/ 初始化；是 Agent 唯一可修改的任务目录，也是 artifacts 指定的提交面。
reference/	专家侧可执行参考解，对 Agent 不可见，只用于证明可解和难度标定。
best_method/	两种专家 Agent 轨迹中，按正式质量门与聚合分数选出且复验成功的唯一最终方法；不得把训练随机 seed 与 Seed Agent 模型名称混为一谈。

grader.py	environment/public_eval/grader.py 为 Agent 可见的 Public/Dev 入口；tests/grader.py 为独立 Verifier 的正式入口。两者对齐指标定义与约束，不采用 Agent 自报结果。
评分与约束校验（Verifier）	专家交付 tests/Dockerfile、实际评分入口及依赖，检查提交格式、硬约束并计算分数。平台在 Agent 结束后移交候选、调度独立 Verifier，并负责正式 Hidden、防作弊和最终验收；无需另交通用调度系统。

❗ 不要继续提交旧版目录或名称：environment/trusted/、tests/runtime/、根目录 baseline/、executions/、额外的 runs/ 中间层、h20_anchor_runs.json、best_policy.py 和单一 trajectory.json 均不属于本版必交结构。

!! 对于 harbor 格式，有不了解的可以参考如下网址：
<https://www.harborframework.com/docs/tasks>，<https://github.com/harbor-framework/harbor>，务必保证符合harbor harness 的兼容性

task.toml

以下为对齐 PCA 内层样例的关键配置摘录。schema_version=1.3 来自该样例，不表示任意 Harbor 版本均支持；实际使用平台指定版本并验证字段生效，任务名称及 metadata 沿用任务卡。资源数值是示例，Agent 与 Verifier 分别按本题需要填写。

代码块

```

1  schema_version = "1.3"
2
3  # 顶层字段：只移交候选及其运行必需产物
4  artifacts = ["/workspace/solution"]
5
6  [environment]
7  cpus = 16
8  memory_mb = 65536
9  storage_mb = 102400
10 gpus = 1
11 build_timeout_sec = 3600
12 network_mode = "no-network"
13
14 [agent]
15 timeout_sec = 43200 # 示例 12h；须容纳原有有效迭代要求及开销
16
17 [verifier]
18 timeout_sec = 3600

```

```

19  environment_mode = "separate"
20  user = "root" # 可信评分进程；候选执行另行降权/隔离
21
22  [verifier.environment]
23  build_timeout_sec = 3600
24  cpus = 8
25  memory_mb = 32768
26  storage_mb = 32768
27  gpus = 0 # 如评测需要 GPU，应明确声明
28  network_mode = "no-network"
29

```

artifacts 只传 /workspace/solution 内符合提交合同的代码、模型及配置，不传 /logs、reward、评分器、Reference 或整个工作区。训练型任务若最终依赖 checkpoint，不能只提交代码而把模型留在未传递的临时路径。提交物中的符号链接、文件大小与冻结文件由可信侧检查。separate 负责最终独立评分，不会提供每轮远程评分接口；仅出现 verifier.environment 小节也不能代替验证独立环境实际生效。

- 结构文档中的常见配置示例为 16 CPU、64 GB、1 张 H20 和 10 MiB 产物上限；它不是所有题目的固定值。
- 示例中的 `metric = "score"` 指归一化分数，因此 `metric_direction` 固定为 `maximize`。本题原始指标的 maximize 或 minimize 方向应在 `instruction.md` 和 `expert_annotation.json` 单独写清。
- 通用资源边界为 CPU 不超过 64C，单个 Job 默认不超过 8 张 GPU，优先使用 H20 或 L20。
- `time_limit_seconds` 应对应单次运行或单次出分预算。题面不写 Agent 的总运行时长；一次“修改、运行、得到 Public/Dev 分数”应尽量不超过 2 小时。
- 若平台模板包含 GPU 卡型、产物大小或并发字段，按任务卡填写，不要自行新增平台无法解析的字段。

instruction.md

`instruction.md` 是 Agent 的任务权威说明。正文必须使用下列八个章节，写清本题的实际规则，不得保留模板注释。

instruction.md 填写要求

```

1  # <Task Title>
2
3  ## Goal
4  [填写：给定什么、产出什么、优化哪个指标及方向；一句话定义合法提交。]
5
6  ## Task Setting
7  [填写：环境、Starter/Baseline、Public/Dev 迭代方式、Hidden 隔离。]

```

```
8
9  ## Objective and Metrics
10 [填写：原始指标、公式、聚合、方向、诊断指标、质量门和分数锚点。]
11
12 ## Allowed Scope
13 [填写：Agent 可修改的准确文件或目录、可读数据和公开运行命令。]
14
15 ## Hard Boundaries
16 [填写：受保护文件、资源/时间/产物限制、禁止访问 Hidden、篡改评分或伪造结果等可执行约束。]
17
18 ## Submission Instructions
19 [填写：最终 solution 的准确文件清单、接口或 Schema，以及 evaluator 如何解析。若任务没有 submission JSON，不要虚构 JSON。]
20
21 ## Workflow & Iteration
22 [填写：修改 → Public/Dev 评分 → 保留或回退 → 更新当前 best 的完整闭环及单轮预算。]
23
24 ## Completion Criteria
25 [填写：接口和质量门通过、产生有限分数、必要产物齐全、受保护内容未改、Hidden 未访问。]
```

以上是简化版的要求，辅助大家理解，大家填写的时候务必要针对一下的完整要求逐条验证：

```
instrument.md完整要求

1  # <Task Title>
2
3  ## Goal
4  <!-- 2-4 sentences: given WHAT, produce WHAT, optimized for WHICH metric and
   direction. Also state what counts as a valid submission in one line. No method
   hints. -->
5
6  ## Task Setting
7  <!-- The environment (sandbox, whether GPU is via launched jobs), the
   starter/baseline the agent begins from, the public dev split for iteration,
   and how the hidden test is kept out of the agent workspace. Note the public
   dev baseline numbers here if one exists. -->
8
9  ## Objective and Metrics
10 <!-- Exact, deterministic scoring: primary metric + formula, aggregation,
   direction (min/max), non-scored diagnostics, and any quality gate. Continuous,
   no clipping, no piecewise anchoring on the reference/baseline. -->
11
12 ## Allowed Scope
13 <!-- What the agent may change/add (e.g. everything under solution/), the
   platform interface scripts it uses (e.g., scripts for launch_job / observe_job
```

```

/ stop_job / cleanup_jobs), and which visible dev/data locations it may read. -
->
14
15 ## Hard Boundaries
16 <!-- Enumerated gates the evaluator enforces: protected files restored at eval
time, artifact/size/time/resource limits, forbidden actions (reading or
peeking at the hidden test, editing the evaluator/metric, fabricating
results), and the requirement that the submission points at the agent's own
run descended from the official scaffold. Concurrency/GPU caps go here. -->
17
18 ## Submission Instructions
19 <!-- The submission file and its schema, required and optional fields, how
submit_job.py writes it, and what the evaluator resolves from it. State that
self-reported scores are not trusted. Include the exact JSON shape. -->
20
21 ## Workflow & Iteration
22 <!-- State that this is an **anytime task**: push performance over days, and
whenever a better solution is found, record it as the current best by
(re)writing the submission – the most recent valid submission is what gets
scored. Give the typical cycle: implement -> launch + observe on public dev ->
iterate if bad -> record current best. Note the per-iteration wall-clock
budget. -->
23
24 ## Completion Criteria
25 <!-- The done checklist: valid schema-conforming submission pointing at the
agent's own successful run, gates pass, a valid score produced, no protected
assets modified, no hidden data accessed. -->
26

```

!

Submission Instructions
必须描述本题真实可执行的提交物：可以是固定文件、字面量配置或 JSON，但不能强制所有题目都使用不存在的 JSON。写明确切文件清单、接口、命令和 evaluator 解析方式；候选自报分数一律不作为正式结果。题面不得直接暴露论文标题、公开解法仓库或其他答案线索，除非任务明确允许。

environment

文件或目录	专家需要完成的内容
Dockerfile	固定基础镜像和依赖，以 environment/ 为 context。只 COPY 本上下文的公开内容，包括 Public/Dev 评分入口；不得复制最终 Verifier 的私有 tests/、Hidden 或 Reference。核心依赖预构建，不靠专家本机临时挂载或内部服务补齐；平台受控的提交物移交与 Hidden 注入另按评分契约执行。

<code>requirements.txt</code>	固定实际依赖及版本。只保留运行必需项，并验证冷启动可安装或已在镜像中准备。
<code>public_assets/train/</code>	可选训练数据。没有训练环节时不创建空的 <code>train/</code> 。
<code>public_assets/dev/</code>	可选 Dev 数据目录，用于每轮检测，不用于训练。允许训练的公开数据放入 <code>train/</code> 或任务卡明确指定的训练目录，并由环境执行边界。即使不使用 <code>dev/</code> 目录，题目也必须提供 Agent 可用的 Public/Dev 评分接口。
<code>starter/</code>	起始实现。构建时从 <code>starter/</code> 初始化 <code>/workspace/starter</code> 和 <code>/workspace/solution</code> ；前者只读，后者按题面可写。Baseline 的定义与运行要求沿用前文。

公开 Dev 反馈要保留。若原来迭代评测也放在 `tests/`，需要将公开部分拆到 `environment/public_eval/`，供 Agent 可见、可调用；最终 Hidden 留在独立 Verifier。公开入口构建到 `/workspace/public_eval/`，原 `solve.sh` 更新为调用这一入口；指标定义与约束同正式评分保持一致，数据切分及对应 B/U 标定按协议分别固定，不把 Dev 分数当成 Hidden 分数。

Dockerfile 的 COPY 最小示意如下；需按本题补齐固定基础镜像、依赖安装、用户及权限。可选数据目录不存在时删除对应 COPY，不创建无用途的空目录。这些片段不替代完整可运行的 Dockerfile。

代码块

```
1 # environment/Dockerfile; context=environment/
2 WORKDIR /workspace
3 COPY starter/ /workspace/starter/
4 COPY starter/ /workspace/solution/
5 COPY public_eval/ /workspace/public_eval/
6 COPY public_assets/ /workspace/public_assets/
7
```

代码块

```
1 # tests/Dockerfile; context=tests/
2 COPY . /tests
3 RUN chmod +x /tests/test.sh
4 RUN mkdir -p /workspace/solution /logs/verifier
5 WORKDIR /workspace/solution
6 # 由 Harbor 在 Agent 结束后移交候选，再执行 /tests/test.sh
7
```

- 验证镜像可以在标准 Harbor 和通用云资源上构建，不依赖内部挂载或内部账号。
- 合并可合并的镜像层，清理包缓存、临时文件和残留数据，并控制镜像体积。

- 验证 Baseline、Reference Solution 和任务级评分校验均能在声明的资源与超时内完成。
- 排查 OOM、FD 泄漏、磁盘写满及日志和产物无限堆积。Agent 不做破坏性操作时，容器应能稳定存活至少12小时；这是环境稳定性目标，与两条Agent轨迹各10小时或7小时例外的有效执行时长分别核验。
- 记录 GPU 型号、张数、CPU、内存、运行时间和 GPU 利用率，并在 `专家作业说明文档.md` 中说明。

solution 与 reference

solution

- `/workspace/solution/` 由 `environment/starter/` 初始化，是 Agent 工作区和唯一候选提交目录。
- 本版标准示例包含 `method.py` + `solve.sh`；若任务确需多文件，必须在 `instruction.md` 明确允许文件和稳定入口。
- `solve.sh` 调用 `/workspace/public_eval/` 下的 Public/Dev 入口，通常冻结不改；不开放 Hidden 调用。最佳方法代码复制到 `best_method/`，结束前将完整最佳候选及必需模型放回 `/workspace/solution/`。

reference

- 专家交付的 reference/ 不进入 Agent 镜像。若评分协议确需参考实现计算真值或对照，可将必要副本作为私有依赖封装在 `tests/`；必须由可信评分器使用，并阻止候选读取。
- Reference 与 Baseline 使用同一数据、协议、资源口径和完整正式 seed 集合。协议是五个 seed 就保留五组，不得压缩为三组代表结果。
- Reference 只证明题目存在稳定优化空间并辅助标定，不是唯一答案，也不是分数上限。

优化面、Baseline 与 Reference 训练证据

本节只要求足以证明优化面成立的最小真实证据：Baseline 与 Reference 在相同正式 seed、数据、训练和评测协议下的逐轮结果、日志、条件性模型产物与统计汇总。数据清单留在 `environment/public_assets/`；方法消融、Agent 候选复测和额外审计属于可选 `expert_evidence/`，不再把大量重复配置拆成独立必交文件。

- ✓ **最低要求：**先声明正式 seed 集合，Baseline 与 Reference 对每个 seed 一一配对、从初始化独立运行。正式协议有五个 seed 就必须完整保留五组，不能只交三组代表结果，也不能改名或用同一 checkpoint 重复评测。训练型任务每轮保留真实模型；非训练型任务不创建空模型目录。

第三个提交目录

`optimization_evidence/` 轻量结构

```

1 optimization_evidence/
2   └─ README.md                # 推荐：统一格式说明
3   └─ 训练证据说明.md          # 必需：本题实例说明
4   └─ baseline_runs/
5     └─ seed_20260911/
6       └─ result.json
7       └─ run.log
8       └─ model/                # 仅训练型任务存在
9         └─ model.pt
10        └─ artifact.json
11        └─ reload.log
12     └─ seed_20260912/
13     └─ seed_20260913/
14     └─ seed_20260914/
15     └─ seed_20260915/
16   └─ reference_runs/
17     └─ seed_20260911/
18     └─ seed_20260912/
19     └─ seed_20260913/
20     └─ seed_20260914/
21     └─ seed_20260915/
22   └─ comparison_summary.json

```

文件或目录	必填内容
<code>README.md</code> （推荐）	定义通用目录、路径基准、状态、哈希、模型条件和汇总规则；避免在每个 seed 中重复整套字段说明。
<code>训练证据说明.md</code>	用中文说明 Baseline/Reference、优化面、正式 seed、固定协议、质量门、复现方式、汇总结论和已知记录缺口。
<code>result.json</code>	一轮主记录，统一包含状态、角色、真实 seed、方法与依赖哈希、协议、训练设置、执行信息、指标、质量门和产物索引。
<code>run.log</code>	原始 stdout/stderr，包括 warning 和失败信息；不得改写成只剩最终分数的人工摘要。
<code>model/</code>	仅训练型任务需要：真实 checkpoint、记录格式/大小/SHA-256/来源和重载结论的 <code>artifact.json</code> ，以及原始 <code>reload.log</code> 。
<code>comparison_summary.json</code>	逐 seed 成对值、两组均值与样本标准差、改善方向、显著性判定、聚合质量门、模型完整性和源结果路径。

💡 **不再作为最小必交项：** `data_manifest.json`、`experiment_plan.json`、`surface_validation.json`、`surface_diff.patch`、`integrity_manifest.json`、`train_config.yaml`、`ablation/` 和额外的 `runs/` 中间层。必要数据指纹放在公开资产 manifest 或 `result.json`；。

专家执行顺序

1. 冻结本题真实 Train 与 Public/Dev 数据版本和切分，生成数据指纹。受许可或隐私限制的数据走平台安全入口，不把未授权原始数据复制进提交包。
2. 运行前在 `训练证据说明.md` 和每轮 `result.json` 中固定正式 seed 列表、指标、协议和资源口径；无需再单独创建 `experiment_plan.json`。Baseline 与 Reference 必须使用完全相同的 seed 集合。
3. Baseline 与 Reference 均从初始化开始独立训练，不能只对同一个 checkpoint 重复评测，也不能从同一个已训练 checkpoint 分叉。保留失败和较差结果，不得挑选性上报。
4. 使用多次训练的均值作为 B/R 锚点，并按“随机性与可分辨性”中的 $3\sigma/5\sigma$ 规则判断 Reference 提升是否超过 Baseline 波动。
5. 训练型任务为每个 seed 保存真实模型，在干净进程加载并复评分；模型路径、格式、大小、SHA-256、方法哈希和重载指标统一写入本轮 `result.json` 与 `model/artifact.json`。
6. 核对 Reference 的实际代码变化只位于题面允许的优化面，并在 `训练证据说明.md` 解释关键差异。多组件方法需要消融时，可将其作为可选附件放入 `expert_evidence/`，不扩张最小证据目录。

非随机波动与优化面有效性的通过口径

检查项	通过要求
运行数量	保留正式协议声明的全部 seed 并逐一配对；若任务尚未规定，Baseline/Reference 至少各 3 次，波动较大时至少 5 次。已经运行五个 seed 就交五组，不抽样。
同口径比较	相同 seed、数据切分、预处理、资源预算、指标和评测协议；B/R 均使用全部有效运行的聚合值。
Maximize 指标	$\text{mean}(\text{Reference}) - \text{mean}(\text{Baseline}) \geq 3\sigma_B$ 。
Minimize 指标	$\text{mean}(\text{Baseline}) - \text{mean}(\text{Reference}) \geq 3\sigma_B$ 。
证据强度	低于 3σ 不通过；达到 3σ 但低于 5σ 时增加重复次数并标记平台复核；达到 5σ 为强证据。
优化面归因	

	Reference 不得修改固定训练/评分内容；说明实际方法差异。多组件消融是可选专家附件，不能替代成对主证据。
模型可复现	训练型任务的每个 checkpoint 均可独立加载并按相同协议出分，seed、role、method hash、文件 SHA-256 和指标一致。

comparison_summary.json 示例 (EPA 五 seed)

```
1  {
2    "schema_version": "autoresearch-comparison-summary.v2",
3    "status": "COMPLETE",
4    "metric": {"name": "hidden_lpips", "direction": "minimize"},
5    "seeds": [20260911, 20260912, 20260913, 20260914, 20260915],
6    "paired_results": {
7      "20260911": {"baseline": 0.157842, "reference": 0.143695,
8        "paired_improvement": 0.014147},
9      "20260912": {"baseline": 0.159233, "reference": 0.145634,
10       "paired_improvement": 0.013599},
11      "20260913": {"baseline": 0.158650, "reference": 0.145815,
12       "paired_improvement": 0.012836},
13      "20260914": {"baseline": 0.164539, "reference": 0.147473,
14       "paired_improvement": 0.017067},
15      "20260915": {"baseline": 0.159531, "reference": 0.146966,
16       "paired_improvement": 0.012565}
17    },
18    "statistics": {
19      "paired_run_count": 5,
20      "baseline_mean": 0.159959,
21      "baseline_sample_std": 0.002640,
22      "reference_mean": 0.145917,
23      "reference_sample_std": 0.001462,
24      "mean_paired_improvement": 0.014043,
25      "relative_improvement": 0.08779,
26      "baseline_sigma_multiple": 5.3186
27    },
28    "significance_rule": {"required_baseline_sigma_multiple": 3.0, "passed":
29      true},
30    "validity": {
31      "same_protocol": true,
32      "same_seed_set": true,
33      "all_quality_gates_passed": true,
34      "all_checkpoints_reload_and_rescore_passed": true
35    }
36  }
```

！ 隔离与真实性： `optimization_evidence/` 只交平台，不进入 Agent 工作区。JSON 内路径统一相对提交包根目录；缺失的命令、时间或退出码必须写 `null` 并解释，不能反推或伪造。模型超过包限制时使用平台认可的不可变资产入口并记录版本、大小和 SHA-256；客观上不训练模型的任务不建空 `model/`，在 `result.json` 标记 `NOT_APPLICABLE`。

tests 与评分校验（格式按需要设置 不固定）

文件	要求
tests/Dockerfile	构建独立 Verifier；context 为 tests/，包含正式评分入口与私有评测资产。 正式评分不能依赖 Agent 容器里临时安装的包、缓存或未提交的文件。Verifier 所需依赖要在自己的镜像里准备好。
tests/grader.py	最终 Hidden 评分入口：校验候选、执行本题推理或必要计算，计算原始指标与归一化分数；不信任候选自报分数。
train_eval.py、rescore_checkpoint.py	按任务需要提供。训练型任务遵循声明的训练/模型加载协议；没有训练步骤时不强制提供。
data.py / metrics.py / score.py / security.py	数据读取、指标、质量门、锚点、约束与执行隔离。内部组织按题目需要，不强制全部文件存在。
test.sh / test_contract.py	/tests/test.sh 在 Agent 结束后实际调用可信 grader 并写 /logs/verifier/reward.txt 或数值对象 reward.json；合同测试可另跑，pytest 成功不能替代最终评分。
hidden_assets/	原教程已有的可选目录。面向 Agent 的公开包不含真实 Hidden；私有 Verifier 构建材料可以按授权预置，或由平台在评分时安全注入。也可在 Verifier 内按固定私有规则生成，无需强制使用这个目录名。

专家负责第二套镜像、任务评分与约束逻辑；平台负责候选移交、独立环境调度和正式验收。Hidden、标签、Blacklist 等通过平台安全入口提交，不放入 Agent 镜像。PCA 样例同时采用私有镜像预置真实数据与评分时生成隐藏数据；本文保留平台安全注入作为可选交付方式，实际选哪一种须写清并实测。

候选代码进入 Verifier 后仍是不可信代码。 可信 grader 可访问私有标签，执行候选时必须限制文件、用户与进程权限；候选只能接收题目允许的测试输入，不能读取私有标签、参考实现，或改写 /tests 与最终 reward。不能把 Python 候选直接 import 到拥有全部私有权限的 grader 进程，就宣称已实现隔离；PCA 使用降权与隔离执行，具体机制可等价实现。

评分锚点与公式

锚点	含义
Baseline B	Starter Code 在正式协议和真实数据上多次独立训练或评测所得原始指标的均值，归一化后为 0；逐次结果放入 <code>optimization_evidence/</code> 。
Reference R	一种合理、可复现的合法改进；R 使用与 Baseline 同一协议下多次独立运行的均值，用于证明可解，不映射到固定分数，也不封顶。
预计可达上限 U	结合已知方法、数据噪声和资源约束作出的 best-effort 估计，归一化后为 1。
Metric Ceiling	指标的数学理论上限。只有无法可靠估计 U 时才用它替代 U。

原始指标越大越好时： $score(x) = \frac{x - B}{U - B}$

原始指标越小越好时： $score(x) = \frac{B - x}{B - U}$

- 评分必须连续、单调、方向明确且不裁剪。Agent 超过 U 时允许分数大于 1。
- 在专家声明的 Public 或 Dev 自验协议上，Reference Solution 的归一化分数应在 0.15 到 0.8 之间，既能证明可解，也要保留优化空间；正式 Hidden 结果由平台复核。
- U 必须与 B 不同，避免归一化公式分母为 0；无法可靠估计 U 时，改用 Metric Ceiling 并记录选择依据。
- 不能在 Reference 或论文结果处改变斜率，不能把论文结果设为分数上限。
- 多目标指标必须写出确切权重、单位、聚合顺序和缺失值处理。

随机性与可分辨性

Baseline 的样本标准差使用： $\sigma_B = \sqrt{\frac{1}{n - 1} \sum_{i=1}^n (B_i - \bar{B})^2}$

- 评测有随机性时，优先遵循任务正式协议声明的完整 seed 列表；协议未规定时，Baseline、Reference 和待评方案至少独立运行 3 次，波动较大时增加到至少 5 次。
- 三者必须使用完全相同的重复次数、seed 集合和聚合方式。已经运行五个 seed 就保留五组，不抽样、不改名；归一化使用全部有效运行的聚合值，不取最好一次。
- Maximize 指标要求 $\bar{R} - \bar{B} \geq 3\sigma_B$ ；Minimize 指标要求 $\bar{B} - \bar{R} \geq 3\sigma_B$ 。
- Reference 到 U 的剩余 Headroom 原则上也不少于 $3\sigma_B$ 。
- 低于 $3\sigma_B$ 时不通过；达到 3σ 但低于 5σ 时可接受，但应增加重复次数并标注供平台复核；达到 5σ 时为强通过。

错误与 Hard Gate

- 成功时输出方向明确的标量分数，并保留可追溯的原始指标。

- 格式错误、Hard Gate、超时、资源超限和基础设施错误必须分类，错误信息要说明可执行的修复动作。
- 非作弊的 Hard Gate 约束违反返回 -1；-1 只表示无效，不参与正常方案排序。格式错误、超时、资源超限和基础设施错误仍须作为不同失败类型返回，不能全部折算为 -1，也不能伪装成 0 分。
- 读取 Hidden Test、篡改评分脚本或伪造结果说明题目隔离失败，专家必须返修环境，不能只靠扣分处理。
- 题面中的每项限制都要由环境或评分脚本实际执行；只写在 `instruction.md` 中不算完成。

数据隔离与防作弊实现

- ☐ Hidden 数据、标签、路径和逐样本结果未进入 Agent 可见镜像、环境变量、日志或共享目录
- ☐ Reference、optimization_evidence/ 和已汇总的 expert_evidence/ 未进入 Agent 可见镜像、缓存或 Git 历史；评分必需的参考副本只在私有 tests/ 中由可信评分器访问，候选不可读取。
- ☐ Agent 结束后只移交 artifacts 指定的最终候选；不复制 Agent 的 /logs、reward 或整个工作区。冻结文件从可信版本校验或恢复。
- ☐ Verifier 从干净结果目录实际评分，由可信进程原子写入最终 reward；候选不能读取私有标签、改写评分器或伪造结果。
- ☐ Git remotes、tags、reflog 和包含答案线索的修复提交已清理
- ☐ 网络、检索工具、可写路径和并发限制已在题面说明，并由环境执行

推荐构建顺序

1. 完成 task.toml 和 instruction.md，固定题目、提交接口、指标和边界；明确 separate 及 artifacts。
2. 搭建 environment/，从 Starter 初始化 /workspace/solution，跑通 Public/Dev 评分与反馈。
3. 完成 tests/Dockerfile、test.sh 和可信评分器，声明 Hidden 预置、生成或安全注入方式；补齐合同测试、错误分类与候选执行隔离。
4. 分别构建 Agent 和 Verifier 镜像，准备独立 reference/。确认公开 Dev 对 Agent 可见、可调用；最终 Verifier 的私有 tests/、Hidden、Reference 及私有评分结果不进入 Agent 环境。
5. 先在 Agent 镜像复验 Baseline 的 Public/Dev 链路；再用平台指定 Harbor 命令完成一次“Agent 结束 → 候选移交 → 独立 Verifier 出分”的端到端自验，核对入口、依赖、路径、资源、reward 和隔离。暂未取得正式 Hidden 权限时，用同接口的授权自验数据验证链路，并明确记录正式 Hidden 待平台复现，不能冒充官方结果。
6. 沿用原要求，在真实数据、完整正式 seed 和相同预算下验证 Baseline 与 Reference；把全部成对证据放入 optimization_evidence/。

💡 `instruction.md` 是 Agent 可读的语义合同，`task.toml` 是机器执行合同，两者必须一致。单 Job 时限、Build Timeout、Setup Timeout、评分超时和 Time-to-one-score 是不同预算，不要用同一个字段替代全部。若任务无法通过缩小模型、数据或训练步数满足两小时出分目标，应记录原因和判据并提交平台复核，不能默默提高资源。

3. 执行 Agent 自迭代

Agent 自迭代用于验证题目可以持续优化。专家需逐轮保留方法迭代记录（轮次、方法名称、方法摘要、执行状态、评分、失败原因、是否保留为最佳、完成时间），并保存最终最佳方法的完整代码；运行编排、详细日志和正式审核记录交由平台处理。

一轮到最终验收的例子：

1. Agent 在 `/workspace/solution/method.py` 修改方法，运行 `bash /workspace/solution/solve.sh`。脚本调用公开 Dev 评分入口，得到本轮指标或失败原因。
2. Agent 根据反馈保留或回退，再继续下一轮；轨迹中的 `score` 始终记录对应 Public/Dev 原始指标。这个过程不调用 Hidden Verifier。
3. 结束前，把经 Public/Dev 复测的最佳候选完整放回 `/workspace/solution`；需要的 `checkpoint` 和配置一并就位。不能只写 `best_method/` 而让 `solution/` 停在最后一次失败实验。
4. 该次 Agent 运行结束后，Harbor 按 `artifacts` 移交最终候选，在独立 Verifier 执行 `bash /tests/test.sh`，产生该提交的 Hidden 分数。Verifier 无需启动 Agent，也不自动给所有历史轮次补 Hidden 分数。

因此，迭代方向由实时 Public/Dev 反馈更新；最终 Hidden 用于独立验收，不能作为本次运行中反复调优的反馈。此时序对齐 PCA 内层 `tests/Dockerfile` 中的 “after the agent exits”。

✅ **本章的最小交付：** 轨迹中每轮按「每轮记录方法迭代字段」一节写全 `round`、`policy_name`、`method_summary`、`status`、`score`、`failure_reason`、`retained_best`、`time` 八个字段；历史代码不随轨迹提交，最终唯一代码交付是 `best_method/`。

模型组合与执行时长

最终轨迹	固定模型组合	默认有效执行时长
<code>trajectory_codex.json</code>	Codex + GPT-5.6 Sol	单独不少于 10 小时
<code>trajectory_seed.json</code>	Codex + Seed 2.1 Turbo	单独不少于 10 小时

 **时长分别计算，不能相加。**默认两个模型组合都要独立运行不少于 10 小时。只有任务不涉及训练或微调、单轮迭代很短，且能够证明“题目可持续 Roll”时，才可缩短为**每个模型组合**不少于 7 小时。

7 小时例外仅适用于不训练、单轮迭代很短的任务，且须同时满足：

1. 两个模型组合都独立运行不少于 7 小时，并有平台运行记录可以核对；排队、环境安装、构建故障和长时间阻塞不计入有效执行时长。
2. 每条轨迹至少完成 3 轮有效闭环：提出并实施方法改动、获得同一 Public/Dev 口径下的评分结论、据此形成下一轮决策。真实失败可以计入，但只排查环境问题不算方法迭代。
3. 方法摘要能够看出连续演进，而不是重复运行同一方案或只产生一两次偶然结果；结束时仍能指出至少一个可继续验证的方向。
4. `best_method/` 已在干净任务快照中重跑并成功出分。
5. 在 `专家作业说明文档.md` 和 `run_summary.json` 中写明两条轨迹的实际有效时长、有效迭代轮数、典型单轮耗时、不涉及训练的依据、单轮迭代很短的运行证据、采用 7 小时例外的理由和上述闭环证据。

模型组合和实际时长属于**运行要求与结果摘要**，不增加到每轮轨迹字段中；每轮轨迹统一记录

`round`、`policy_name`、`method_summary`、`status`、`score`、`failure_reason`、`retained_best`、`time` 八个字段，模型组合与有效时长写入 `run_summary.json` 和专家说明。

运行前准备

1. 从干净的 Agent 镜像运行未修改的 Starter Code，确认 Baseline 可获得 Public/Dev 分数；完整 `harbor_task/` 只由平台读取，不整体挂载给 Agent。
2. 分别启动 Codex + GPT-5.6 Sol 和 Codex + Seed 2.1 Turbo；两者使用同一版本的任务、Public/Dev 评分口径和资源配置。
3. 每次启动使用独立的空输出目录，不向 Agent 暴露 Reference、Hidden Test、参考分数或另一个 Agent 的轨迹。

启动 Agent

使用任务卡提供的 Harbor Harness 命令启动。Codex + GPT-5.6 Sol 和 Codex + Seed 2.1 Turbo 使用同一份 Prompt、独立输出目录和相同任务配置；推理档位默认最大。

Agent 自迭代 Prompt

- 1 你是一个负责长程算法优化的 Autoresearch Coding Agent。
- 2
- 3 任务目录：

```
4 TASK_ROOT=/workspace
5 OUTPUT_DIR=/expert_evidence
6
7 请先阅读 Harness 提供的 instruction.md 任务说明与资源约束，完整运行未修改的 Baseline，
  随后只在允许范围内迭代方法，并以任务规定的 Public/Dev 评分结果判断是否改进。
8
9 每完成一轮：
10 - 向 ${OUTPUT_DIR}/trajectory.jsonl 追加一行本轮记录，依次包含 round、policy_name、
    method_summary、status、score、failure_reason、retained_best、time；
    method_summary 用 1 至 3 句话说清本轮尝试了什么、得到什么结论，score 以评分脚本实际输出
    为准、未成功出分记 null，本轮方法刷新当前最佳时 retained_best 记为 true，time 记本轮完
    成时间（YYYY-MM-DD HH:mm:ss）。
11 - 如果本轮产生了当前最佳的有效方法，将其完整可运行代码更新到
    ${OUTPUT_DIR}/best_method/。
12
13 不得尝试访问未公开评测资产或参考实现，不得修改 solution/ 以外的任务文件，不得篡改评分脚本
    或伪造结果。
14
15 每个模型组合独立有效迭代至少10小时；仅当任务不涉及训练、单轮迭代很短且本教程例外证据全部满
    足时，可缩短为每个模型组合至少7小时。排队、安装、构建故障和长时间阻塞不计入有效时长；不另
    设统一的大于15轮门槛。结束前重新运行当前最佳方法，确认其能够正常运行并获得 Public/Dev 分
    数；将该方法及全部必需产物恢复到 /workspace/solution。最终 Hidden 评分由 Harbor 在本
    次 Agent 运行结束后调用独立 Verifier 完成，不作为本次迭代反馈。
```

OUTPUT_DIR 是本次 Agent 运行专用的空白输出位置，仅用于新产生的轨迹和最佳代码；不得把已汇总的 expert_evidence/、其他 Agent 轨迹或优化证据包挂载进去。平台提供该输出位置的写权限，它不属于需要传给 Verifier 的 artifacts。

每轮记录方法迭代字段

字段	专家需要确认的内容
round	从 1 开始递增，用于保持方法演进顺序。
policy_name	本轮方法或策略名称，与 method_summary 对应。
method_summary	用 1 至 3 句话写明本轮的核心改动和结论。失败时可直接在摘要中说明原因及下一步判断。
status	本轮执行状态，成功记为 ok；失败时按真实失败类型填写，并在 failure_reason 中说明。
score	本轮在对应 Public/Dev split 上得到的原始指标值，以评分脚本实际输出为准，不采用自报结果；本轮未成功出分时记 null。

failure_reason	失败时简要说明原因及下一步判断；成功时记 <code>null</code> 。
retained_best	标记本轮方法是否为至今保留的最佳方法，取布尔值 <code>true</code> / <code>false</code> 。
time	本轮完成时间戳，统一使用 <code>YYYY-MM-DD HH:mm:ss</code> 格式的字符串。

trajectory.jsonl 单行示例

```
1  {
2    "round": 1,
3    "policy_name": "cosine-annealing-lr",
4    "method_summary": "将固定学习率改为余弦退火；验证结果优于上一轮，因此保留该方向。",
5    "status": "ok",
6    "score": 0.23,
7    "failure_reason": null,
8    "retained_best": true,
9    "time": "2026-09-10 16:20:32"
10 }
```

💡 Agent 轨迹每轮需记录 `round`、`policy_name`、`method_summary`、`status`、`score`、`failure_reason`、`retained_best` 和 `time`；运行期 `trajectory.jsonl` 每行一个轮次对象，整理时按顺序放入最终轨迹 JSON 的 `rounds` 数组。轨迹中不要求 run ID、运行时长、命令、退出码、seed、资源占用或模型哈希；模型组合与实际有效时长统一写入 `run_summary.json` 和专家说明，不增加到轨迹字段。但 Baseline/Reference 的 seed、配置、命令、训练日志、指标及模型哈希是 `optimization_evidence/` 的必交内容，不得因轨迹简化而省略。

两种模型组合分别整理

Codex + Seed 2.1 Turbo

按实际轮次顺序逐轮记录完整方法迭代字段，最终整理为 `trajectory_seed.json`。

Codex + GPT-5.6 Sol

使用独立空输出目录，不读取另一模型组合的轨迹或最佳解；最终整理为 `trajectory_codex.json`。

时长按两个模型组合分别核算；如当前任务卡还要求多次独立运行，则同时满足任务卡要求。无论运行多少次，专家最终提交的轨迹都按顺序保留每轮完整字段：轮次、方法名称、方法摘要、状态、评分、失败原因、最佳标记和完成时间。

整理最终最佳方法

- 1. 在同一 Public/Dev 评分口径下比较两个模型组合产生的有效方法，选择归一化分数最高的方案。

2. 将最终方案的全部可运行代码、入口和所需配置放入 `expert_evidence/best_method/`，不需保留历史版本。
3. 在干净任务快照中，用 `best_method/` 替换 `/workspace/solution/` 后重跑 Public/Dev 评分。如果无法重跑，修复后再交付，或改交上一份可运行方法。

每次 Agent 运行交给 Verifier 的对象是它结束时 `/workspace/solution` 的完整候选。`best_method/` 仍只交最终代码；训练型候选用于评分的模型按任务合同放入 `/workspace/solution`。超限时使用平台批准的不可变产物入口，由平台在评分前解析为本地可加载资产，并记录加载与移交方式，不另加每轮镜像或评分服务。

4. 整理轨迹与专家证据

完成 Codex + GPT-5.6 Sol 和 Codex + Seed 2.1 Turbo 两条独立自迭代、确认最佳方法可重跑后，整理最终专家出题包。本次提交包含三个相互隔离的顶层目录：可执行题目、专家交付，以及证明优化面与参考解真实有效的训练证据。

最终交付结构

同一次提交的三个顶层目录

```
1  workspace/
2  |— harbor_task/
3  |— reference/
4
5  expert_evidence/
6  |— README.md                # 推荐
7  |— 专家作业说明文档.md
8  |— best_method/
9  |— expert_annotation.json
10 |— run_summary.json
11 |— trajectory_seed.json
12 |— trajectory_codex.json
13
14 optimization_evidence/
15 |— README.md                # 推荐
16 |— 训练证据说明.md
17 |— baseline_runs/
18 |   |— seed_<真实seed>/{result.json, run.log, model/}
19 |— reference_runs/
20 |   |— seed_<真实seed>/{result.json, run.log, model/}
21 |— comparison_summary.json
```

交付物	唯一用途
workspace/	可执行题目：含两套环境的完整 harbor_task/ 与专家侧 Reference；公开内容及 Public/Dev 入口构建到 Agent 镜像，最终 Verifier 的私有 tests/ 和 Reference 不向 Agent 暴露。
专家作业说明文档.md	让平台和复现人员理解任务、运行、评分、隔离、专家过程和已知限制。
best_method/	两条专家 Agent 轨迹中选出的唯一最终方法代码，不放训练 checkpoint。
expert_annotation.json	题目类型、优化面、指标方向、B/R/U 和隔离关系的精简机读标注。
run_summary.json	专家自验状态、两种 Agent 模型组合的有效时长、复测和选优依据。
trajectory_seed.json / trajectory_codex.json	分别保存两种 Agent 模型的逐轮完整迭代记录（八个字段）；不混入 Baseline/Reference seed 证据。
optimization_evidence/	只保存 Baseline/Reference 的全部正式 seed 结果、原始日志、训练型模型与统计汇总。

 **三个目录职责不重复：** workspace/ 是可执行题目； expert_evidence/ 是专家标注、Agent 轨迹和最终方法； optimization_evidence/ 是 Baseline、Reference 与优化面的独立真实性证据。

专家作业说明文档

说明文档用于让平台和复现人员不靠猜测即可运行题目。建议按以下顺序填写。

1. Task ID、题目名称、S1/S2/S3、研究方向和变体编号。
2. 问题定义：输入、输出、搜索空间、允许修改范围和固定内容。
3. 构建与运行：两套 Dockerfile、各自 context、构建命令与镜像标识，以及 Public/Dev 和最终 Verifier 入口；记录目标 Harbor 版本、separate/artifacts 生效及候选移交自验证据。沿用 Baseline、Reference、多次独立训练、模型加载与复评分说明。
4. 评分设计：原始指标、方向、B/R/U、归一化公式、随机评测口径和资源；B/R 必须使用多次训练均值，并引用 optimization_evidence/comparison_summary.json 的 3 σ /5 σ 证据。
5. 隔离设计：Agent 可见、可写和不可见内容；Hidden 预置、生成或安全注入方式，以及候选在 Verifier 中的受限执行方式。明确是否已获授权实跑 Hidden。
6. 自迭代简要结论：Codex + GPT-5.6 Sol 和 Codex + Seed 2.1 Turbo 的实际有效时长、尝试的方法方向，以及最终选择 best_method/ 的理由。若使用 7 小时例外，先说明任务不涉及训练、单轮迭代很短的依据，再写明每条轨迹至少 3 轮有效闭环等可持续 Roll 证据。

7. 已知限制与风险。只陈述专家自验事实，不写“审核通过”或官方 Hidden 结果。

expert_annotation.json

 `expert_annotation.json` 是一份**机器可读的题目标注摘要**，不是另一份专家说明，也不是运行日志。平台用它在`不解析长文`的情况下，识别题目类型、指标方向、标定锚点和基本隔离声明。

内容	是否放入
题目语义和量化口径	放入：Task ID、模式、研究方向、优化面、指标方向、B/R/U 和实际 split。
基本隔离声明	放入：Reference 和 Hidden 资产是否向 Agent 隐藏。
运行过程	不放入：不写每轮方法、run ID、时间戳、日志、资源或失败枚举。
审核结论	不放入：不写 <code>approved</code> 、 <code>audit_passed</code> 或官方 Hidden 分数。

```
expert_annotation.json 最小结构
1  {
2    "task_id": "<platform_task_id>",
3    "task_type": "<S1_or_S2_or_S3>",
4    "research_direction": "<research_direction>",
5    "optimization_surface": "<optimization_surface>",
6    "score_contract": {
7      "metric": "accuracy",
8      "direction": "maximize",
9      "split": "dev",
10     "protocol": "<protocol_id_or_version>",
11     "baseline": {
12       "raw_value": 0.55,
13       "normalized_score": 0.0
14     },
15     "reference": {
16       "raw_value": 0.73,
17       "normalized_score": 0.72
18     },
19     "upper_bound": {
20       "raw_value": 0.80,
21       "source": "estimated_attainable_upper_bound"
22     }
23   },
```

```

24     "isolation": {
25         "reference_visible_to_agent": false,
26         "hidden_assets_visible_to_agent": false
27     }
28 }

```

- 示例数值只展示类型和计算关系，必须替换为本题的真实锚点。
- `direction` 填原始指标方向；归一化分数仍始终是越高越好。
- Baseline、Reference 和 U 使用同一 split 和协议。无法可靠估计 U 时，将 `source` 改为 `metric_ceiling`。
- Baseline 和 Reference 的锚点必须来自同一协议下多次独立训练的汇总值，不得取单次最好结果；逐次结果和统计口径放入 `optimization_evidence/comparison_summary.json`。
- 如平台后续下发固定 Schema，以平台模板为准，不在教程示例上自行增加字段。

run_summary.json

`run_summary.json` 记录本次专家自验的最终状态、两种固定模型组合的实际有效时长、7 小时例外证据，以及第三个目录是否备齐；不重复每轮轨迹、训练明细、评分锚点或平台原始日志。

run_summary.json 最小结构

```

1  {
2      "task_id": "<platform_task_id>",
3      "status": "pilot_required",
4      "image": "<local_image_name_or_tag>",
5      "agent_execution": [
6          {
7              "runtime": "Codex",
8              "model": "GPT-5.6 Sol",
9              "duration_hours": 10.4,
10             "effective_rounds": 9,
11             "typical_round_minutes": 42
12         },
13         {
14             "runtime": "Codex",
15             "model": "Seed 2.1 Turbo",
16             "duration_hours": 10.2,
17             "effective_rounds": 11,
18             "typical_round_minutes": 35
19         }
20     ],
21     "fast_iteration_exception": {
22         "used": false,
23         "non_training": null,

```

```

24     "short_iterations": null,
25     "rollable_proved": false,
26     "evidence": ""
27 },
28 "validation": {
29     "baseline_passed": true,
30     "reference_passed": true,
31     "optimization_evidence_complete": true,
32     "best_method_rerun_passed": true
33 }
34 }

```

`duration_hours` 填各模型组合的实际有效执行时长，分别核算、不得相加；
`effective_rounds` 和 `typical_round_minutes` 说明真实迭代节奏。默认两项时长都不得低于 10。只有任务完全不涉及训练或微调、单轮迭代很短，且两项时长均不少于 7 并满足全部可持续 Roll 条件时，才可将 `used`、`non_training`、`short_iterations` 和 `rollable_proved` 设为 `true`；`evidence` 须说明无训练的依据、典型单轮耗时及闭环证据。训练型任务即使迭代很快，也仍须各不少于 10 小时；不另设统一的单轮分钟数阈值。未使用例外时，两个资格字段可为 `null`。只有 Baseline/Reference 按任务类型完成正式运行和非随机增益验证，训练型另完成规定的重复训练及模型重载时，`baseline_passed` 与 `reference_passed` 才能为 `true`。`optimization_evidence_complete` 仅表示专家已备齐第三个目录，不代表平台审核通过；各项自验任一为 `false` 时，先修复再提交。

为保持原有 `run_summary.json` Schema，`image` 字段仍填写 Agent 镜像；Verifier 镜像的名称/不可变标识及双镜像实跑证据统一写入 专家作业说明文档.md，不得擅自增加平台未接收的字段。两套镜像仅构建成功不等于正式 Hidden 验收通过。

trajectory_seed.json 与 trajectory_codex.json

两份文件的结构完全相同：`trajectory_seed.json` 对应 Codex + Seed 2.1 Turbo，`trajectory_codex.json` 对应 Codex + GPT-5.6 Sol。文件内每轮记录 `round`、`policy_name`、`method_summary`、`status`、`score`、`failure_reason`、`retained_best`、`time` 八个字段；模型组合与实际有效时长统一写入 `run_summary.json` 和专家说明，不另增轨迹字段。

最终轨迹最小结构

```

1  {
2    "rounds": [
3      {
4        "round": 1,
5        "policy_name": "mixed-relative",
6        "method_summary": "将固定学习率改为余弦退火；验证结果优于上一轮，因此保留该方向。",

```

```

7      "status": "ok",
8      "score": 0.23,
9      "failure_reason": null,
10     "retained_best": true,
11     "time": "2026-09-10 16:20:32"
12 },
13 {
14     "round": 2,
15     "policy_name": "mixed-relative",
16     "method_summary": "增加了额外正则项；收益不稳定，下一轮回退该改动。",
17     "status": "ok",
18     "score": 0.21,
19     "failure_reason": null,
20     "retained_best": false,
21     "time": "2026-09-10 16:27:32"
22 }
23 ]
24 }
```

- `round` 按实际迭代顺序从 1 递增，每轮一个对象。
- `policy_name` 记录本轮方法或策略名称，与 `method_summary` 对应。
- `method_summary` 用 1 至 3 句话写清本轮改了什么、得到什么结论；不要在其中粘贴大段代码或原始日志。
- `status` 记录本轮执行状态（如 `ok`）；失败时用 `failure_reason` 简要说明原因，成功时为 `null`。
- `score` 填本轮对应 Public/Dev split 的原始指标值，不混入最终 Hidden 分数；
`retained_best` 标记该轮是否为至今保留的最佳方法。
- `time` 记录本轮完成时间戳。
- 轨迹中不保存历史代码；最终代码不放入轨迹，只放在 `best_method/`。
- 运行期 `trajectory.jsonl` 每行是一个轮次对象；整理时按顺序放入 `rounds` 数组，形成完整 JSON。

best_method/ 交付要求

- ☐ 仅保留最终最佳方法，不保留历史快照或两份候选方法
- ☐ 包含入口脚本、所需源码和运行必需配置，可替换容器内 `/workspace/solution/` 的方法代码；需要模型时另按提交合同确保评分可加载。
- ☐ 不包含 Reference、Hidden 数据、评分日志、运行 ID、审核材料或密钥
- ☐ 已在干净任务快照中重新运行并成功出分

提交前专家自查

- ☐ Repo、`expert_evidence/` 与轻量 `optimization_evidence/` 三类交付均完整，职责不重复
- ☐ 两套 Dockerfile 分别以 `environment/` 与 `tests/` 为 context；公开评分位于 Agent 构建上下文，正式评分位于 `tests/`；没有越界 COPY 或把完整任务包挂载给 Agent。
- ☐ `instruction.md` 已写清问题、评分和可修改边界
- ☐ Starter、Baseline、Reference 和任务级评分校验均可运行
- ☐ `optimization_evidence/` 保留正式协议声明的全部真实 seed；Baseline/Reference 逐 seed 配对、显著性规则成立，训练型任务每轮都有可加载模型
- ☐ `expert_annotation.json` 只包含精简机读标注，未混入运行日志或审核结论
- ☐ `run_summary.json` 已记录两个固定模型组合的实际有效时长，且各项专家自验均为 `true`
- ☐ Codex + GPT-5.6 Sol 和 Codex + Seed 2.1 Turbo 均独立运行不少于 10 小时；如缩短至不少于 7 小时，已确认任务不涉及训练且单轮迭代很短，并提交完整例外证据
- ☐ 两份轨迹都是合法 JSON，且每轮完整包含 `round`、`policy_name`、`method_summary`、`status`、`score`、`failure_reason`、`retained_best`、`time` 八个字段
- ☐ `best_method/` 是唯一最终代码实现，且可从干净环境重跑
- ☐ Agent 可见内容不含 Hidden、Reference、`optimization_evidence/`、已汇总的 `expert_evidence/` 完整证据包、专家本机路径或密钥；本次运行的空白输出目录仅收集新产物，所有结果均为真实运行所得。
- ☐ 题面包含完整八个章节，无方法提示，无论文标题 / arXiv ID / repo 名称等泄露信息
- ☐ 目标 Harbor 版本可解析本题配置，`separate` 和 `artifacts` 实际生效；两套镜像均已构建成功，并保留各自镜像标识。
- ☐ 每轮 Public/Dev 反馈可用于继续迭代；结束时最佳候选及所需模型在 `/workspace/solution` 就位。
- ☐ 已验证 Agent 结束后仅移交指定候选，独立 Verifier 实际执行 `/tests/test.sh` 并出分；授权自验数据与正式 Hidden 结果明确区分。
- ☐ 独立 Verifier 中的候选不能读取私有标签/参考实现或写入可信评分器与最终 reward；未把两个容器本身当成全部隔离证明。
- ☐ 评分函数连续、单调、不裁剪，`baseline = 0`（在 hidden test 上）、预计可达上限 = 1
- ☐ 参考解分数不低于 0.15 且不高于 0.8，且可以复现
- ☐ 对于有波动性的 verifier，额外要求参考解与 Baseline 的可分辨性：`maximize` 指标要求 $\bar{R} - \bar{B} \geq 3\sigma$ ；`minimize` 指标要求 $\bar{B} - \bar{R} \geq 3\sigma$

- ☐ 所有题面 constraints 均由环境或 verifier 代码强制 enforce，非仅文字声明
- ☐ Verifier 单命令执行，成功输出标量分数，各类失败有明确区分的错误信息
- ☐ Hidden 数据不出现在 Agent 可见的镜像、任务包、环境变量、日志、共享目录中
- ☐ 参考解代码及结果不出现在 Agent 可见的镜像、任务包、环境变量、日志、共享目录或 Git 历史中
- ☐ Verifier 启动时清理 Agent 预写结果文件，由 trusted verifier 原子写入
- ☐ Evaluator / guard / metric / baseline / 参考解等冻结面在评估时从可信版本恢复，参考解不进入 Agent 可见工作区
- ☐ `instruction.md` 符合教程文档中的 8 项要求
- ☐ Git 仓库已清理 remotes / tags / reflog，无 fix commit 等答案线索
- ☐ Baseline 能跑出 non-trivial 分数（非随机波动），参考解证明分数可提升
- ☐ 至少保留一种题面未直接提示但合法可发现的改进方向，参考方法未在 verifier 上饱和
- ☐ 已附 ≥ 1 条验证过的 agent 轨迹 / 参考解及执行日志，并完成 trajectory analysis
- ☐ 若评估有随机性，已声明 seed / replicate 规则，有效提升阈值（如 3σ ）已在题面注明
- ☐ 题面已明确可修改范围、网络规则、工具策略（webSearch 等）
- ☐ `trajectory_seed.json`、`trajectory_codex.json`、`expert_annotation.json`、`run_summary.json` 符合填写规范（重点检查 highlight 项目）

☒ 平台收包后负责正式复现、Hidden Test、防作弊审核和最终验收。专家不需要交平台内部运行日志、审核目录或“审核通过”结论，但必须提交自己实际产生的 Baseline/Reference 训练记录、评测结果和模型。

附录 A | PCA 例题按本教程格式整理

本节对应文末附件 `population-genetics-pca_teaching-format_20260929.zip`。下载后先读包根 README.md，再看 expert_evidence/专家作业说明文档.md；题面已重组为本教程八章节，目录按三类交付整理。

示例定位：可以照着学习目录、公开迭代与最终双镜像评分；不是已通过全部题目门槛的验收样例。原 PCA 的 Hidden 评分数学保留；公开 Starter/Dev 是本轮新增的教学材料，未把历史 README 分数挪作新题成绩。尚未运行的轨迹和正式证据明确留空，不能原样当作完整正式交付。

A.1 题目做什么，哪些文件分别负责什么

输入多样本 VCF，输出按样本顺序排列的前 k 个主成分。方法需要遵守 HWE 标准化定义，在正确性约束下改善 CPU 时间和内存使用。本题不训练模型，不需要 checkpoint 或空 model/ 目录。

角色	本例文件与职责
Starter / Baseline	environment/starter/pca + pca_core.py：完整扫描实现，已适配成独立候选；作为公开起点。
Public/Dev	environment/public_eval/grader.py：在 Agent 环境每轮调用；读取3个公开小型VCF，输出真实代理分数和诊断。
专家参考解	workspace/reference/fast_pca.py； fast_submission/pca 是其可提交版本，不复制给 Agent。
最终 Verifier	tests/Dockerfile + tests/test.sh + tests/grader_pkg/：Agent结束后独立评分，输出正式 native_reward。
Agent 最佳方法	expert_evidence/best_method/：须由真实迭代选优后产生；当前只有待补说明，未用 Reference 冒充。

原包外层 workspace/submission/pca 与快速参考解相同，因此不能直接拿来当 Starter。本例用完整扫描实现作为起点；这改变了原“从空目录实现”的初始难度，历史成绩不再能直接比较。

A.2 最终目录

代码块

```
1  pca_teaching_example/
2  |— README.md
3  |— workspace/
4  |   |— harbor_task/
5  |   |   |— instruction.md           # 八章节题面
6  |   |   |— task.toml               # separate + artifacts
7  |   |   |— environment/           # Agent build context
8  |   |   |   |— Dockerfile
9  |   |   |   |— requirements.txt
10 |   |   |   |— starter/{pca,pca_core.py}
11 |   |   |   |— public_eval/         # grader.py、公开数据生成脚本
12 |   |   |   |— public_assets/dev/   # 3个公开VCF及manifest
13 |   |   |   |— tests/               # Verifier build context
14 |   |   |   |   |— Dockerfile
15 |   |   |   |   |— test.sh
```

```

16 | | | └─ grader_pkg/
17 | | |   └─ grader/           # 正式评分和候选隔离
18 | | |   └─ data/           # Hidden派生/生成
19 | | |   └─ reference/      # 评分所需私有参考副本
20 | | └─ reference/         # 专家侧主参考解
21 | └─ expert_evidence/
22 |   └─ 专家作业说明文档.md
23 |   └─ expert_annotation.json
24 |   └─ run_summary.json
25 |   └─ trajectory_seed.json # 当前rounds=[]
26 |   └─ trajectory_codex.json # 当前rounds=[]
27 |   └─ best_method/README.md # 待真实Agent选优
28 |   └─ validation/         # 本轮真实检查记录
29 └─ optimization_evidence/
30   └─ 训练证据说明.md
31   └─ baseline_runs/README.md # 待正式配对实验
32   └─ reference_runs/README.md
33   └─ comparison_summary.json # NOT_RUN, 统计量null
34

```

本例保留 `pca` 作为程序入口，没有强行改名为 `method.py + solve.sh`；`tests` 内部保留 `grader_pkg` 模块组织。这两处符合本教程“内部实现按优化面设置”的要求。

完整包只交专家/平台，不整体给 Agent。公开镜像只使用 `environment/`。`workspace/reference` 是专家交付；`tests/grader_pkg/reference` 是正式评分需要的私有依赖，可存在于 Verifier 内，但 Agent 和执行中的候选均不可读取。

A.3 从一次迭代到最终评分

1. 构建 Agent 镜像，由 `starter/` 初始化容器内 `/workspace/solution`；公开评分和公开数据只读。
2. Agent 修改 `solution/pca` 或其辅助模块，运行下面的 Dev 命令，读取 JSON 中的 `status`、`score` 和逐例诊断。
3. 若质量门通过且同协议复测更好，保留方法；否则回退或继续调整。轨迹 `score` 记录实际 Public/Dev 代理指标，不填最终 Hidden 分数。
4. 结束前把 Dev 最佳候选恢复到 `/workspace/solution` 并复测。只在 `best_method/` 留一份备份不能保证 Harbor 拿到它。
5. Agent 结束后，Harbor 按 `artifacts=["/workspace/solution"]` 移交候选，在独立 Verifier 中执行 `/tests/test.sh`。Verifier 执行评分程序及受限候选，不启动第二个 Coding Agent。
6. 最终写 `/logs/verifier/reward.json`。不会自动给历史轮次补 Hidden 分数，也不把本次最终 Hidden 结果回传作继续调参反馈。

```
1 # 容器内：每次方法修改后运行
2 python /workspace/public_eval/grader.py \
3     --submission /workspace/solution --repeats 3
4
```

Dev 指标为各公开 case 的 $\log_2$ (基线中位运行时间/候选中位运行时间) 的均值，越高越好，不裁剪；每次重复均要求子空间相似度 ≥ 0.98 ，样本顺序、输出维度与有限性等检查通过。失败返回分类状态和 `score=null`。它是透明的教学代理指标，与原生 Hidden 分数不等价，也不保证排名一致。

A.4 构建与路径对照

在解压后的 `pca_teaching_example/` 根目录执行：

```
代码块
1 docker build -t pca-teaching-agent:local \
2     -f workspace/harbor_task/environment/Dockerfile \
3     workspace/harbor_task/environment
4
5 docker build -t pca-teaching-verifier:local \
6     -f workspace/harbor_task/tests/Dockerfile \
7     workspace/harbor_task/tests
8
```

两侧保留原题8 CPU、16 GiB内存、0 GPU及固定依赖。Verifier 构建期需要网络获取约206 MB、校验固定的压缩真实数据并派生私有资产；运行期无网络。本轮只将候选路径统一成 `/workspace/solution`，并将最终 `reward.json` 改为原子写入，未重写私有评分数学与隔离机制。

A.5 已验证什么，还需补什么

项目	本例状态
源码/目录/配置	双context、候选路径、TOML、Python/Shell语法及私有评分源码保留关系已检查。
公开接口	Starter通过3个小型Dev用例，实际smoke原始记录已收录validation/；独立小VCF数值、错误输出和缺失入口检查的观察说明另行列出。本地依赖与固定Docker版本不同，不能当作镜像验收。
最终reward写入	有限数值校验和原子输出片段已用合成输入验证，未执行完整Hidden。
Docker/Harbor/Hidden	本机Docker daemon未启动；镜像构建、完整移交、Linux隔离及正式Hidden仍待目标环境实跑。

专家轨迹/最佳方法	未跑教程要求的两套长程Agent；rounds=[], best_method仅待补说明，时长为null。
成对优化证据	原包没有可迁移的完整seed结果和日志；comparison_summary为NOT_RUN，不伪造seed目录、统计量或model/。
统一评分要求	原native_reward有[0,1]截断/分段，Reference用于速度锚点，部分无效提交为failed/0；尚未对齐B=0、Reference∈[0.15,0.8]、不裁剪及统一失败语义。正式化前须改造、重标定并补证。

相同Baseline代码在小样本计时中也可能获得非零代理分数，原因包括启动、缓存和执行顺序；不能把一次smoke的正值解释成方法提升。本例用于教会专家“文件放哪里、每轮怎么跑、最终怎么交”，研究质量与正式验收仍按正文门槛执行。

A.6 示例包下载

下方ZIP包含实际源码、公开Dev数据、私有Verifier与诚实填写的证据模板；不包含已实跑的Docker镜像或正式Hidden成绩。



population-genetics-pca_teaching-…

299.32KB

